



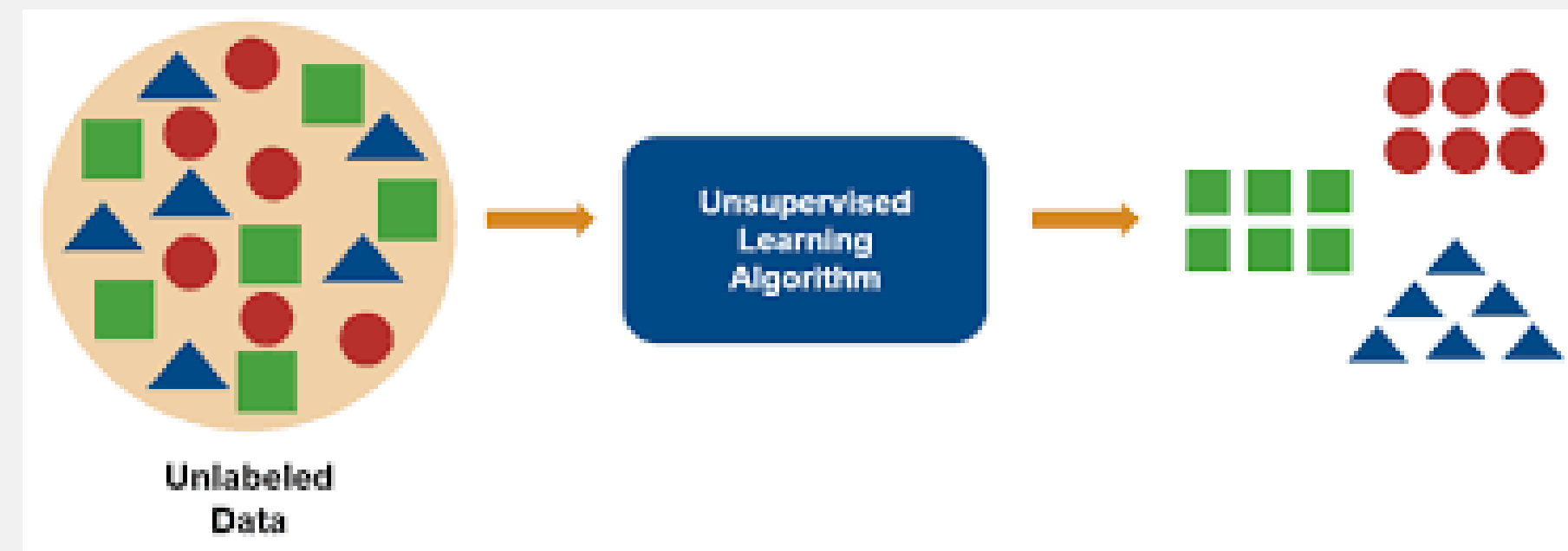
Unsupervised Learning

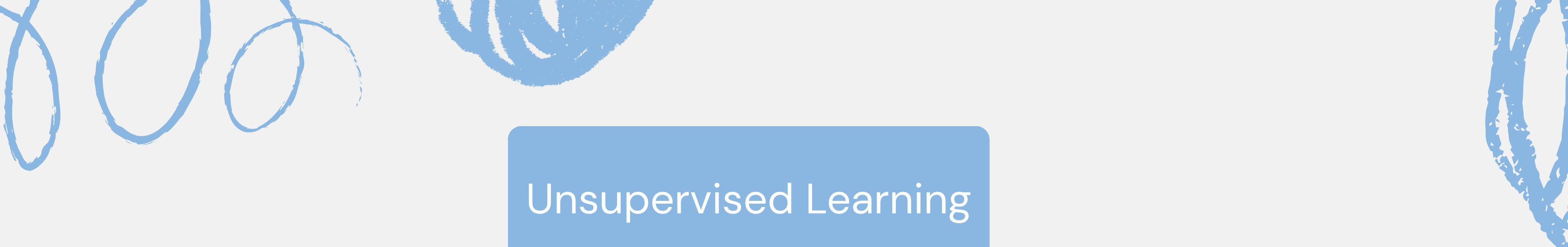
MSP'26 ML Module Session04



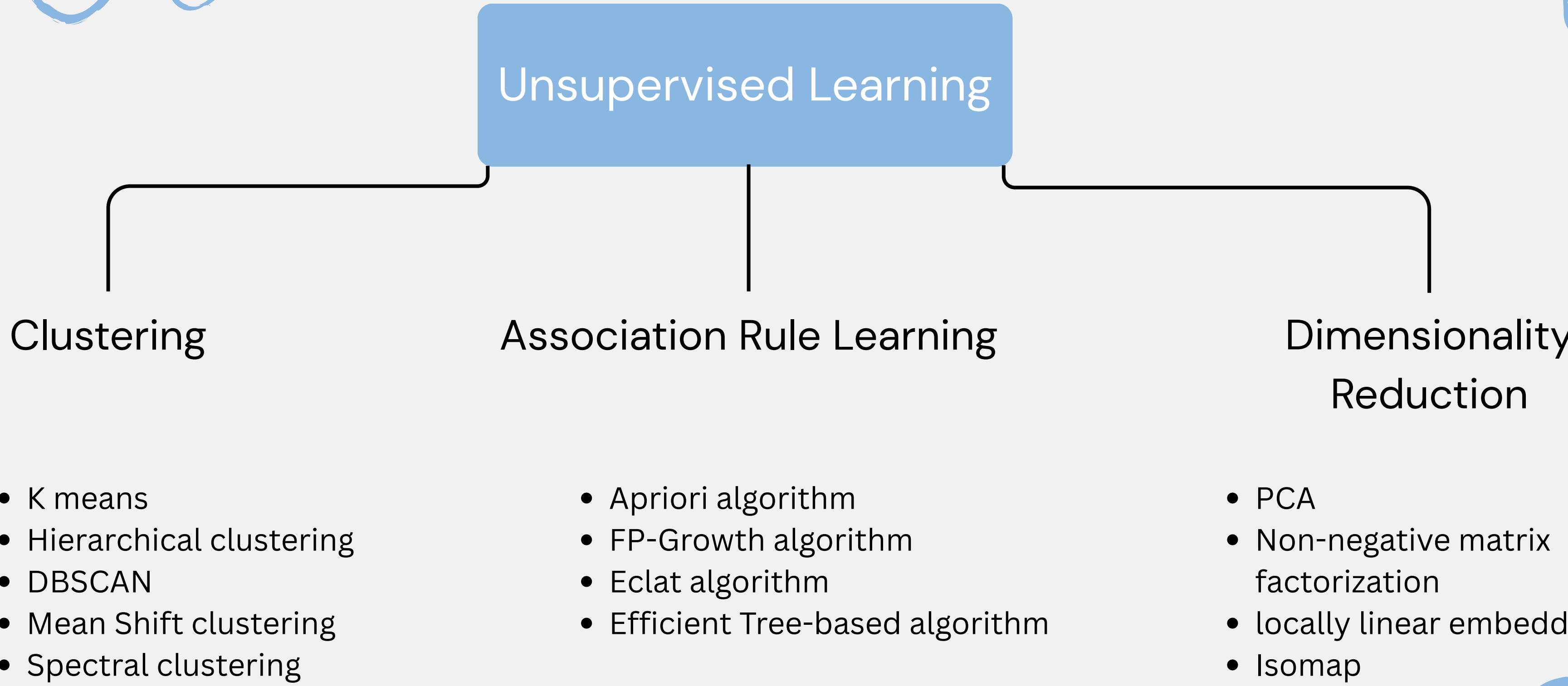
Unsupervised Learning

Unsupervised Learning is a type of machine learning where the model works without labelled data. It learns patterns on its own by grouping similar data points or finding hidden structures without any human intervention.





Unsupervised Learning



```
graph TD; A[Unsupervised Learning] --> B[Clustering]; A --> C[Association Rule Learning]; A --> D[Dimensionality Reduction]; B --> B1[K means]; B --> B2[Hierarchical clustering]; B --> B3[DBSCAN]; B --> B4[Mean Shift clustering]; B --> B5[Spectral clustering]; C --> C1[Apriori algorithm]; C --> C2[FP-Growth algorithm]; C --> C3[Eclat algorithm]; C --> C4[Efficient Tree-based algorithm]; D --> D1[PCA]; D --> D2[Non-negative matrix factorization]; D --> D3[locally linear embedding]; D --> D4[Isomap];
```

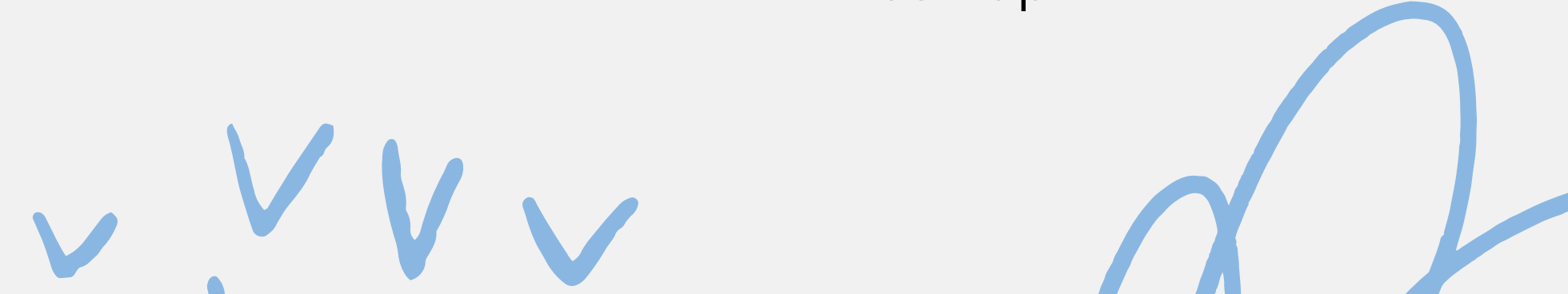
Clustering

- K means
- Hierarchical clustering
- DBSCAN
- Mean Shift clustering
- Spectral clustering

Association Rule Learning

- Apriori algorithm
- FP-Growth algorithm
- Eclat algorithm
- Efficient Tree-based algorithm

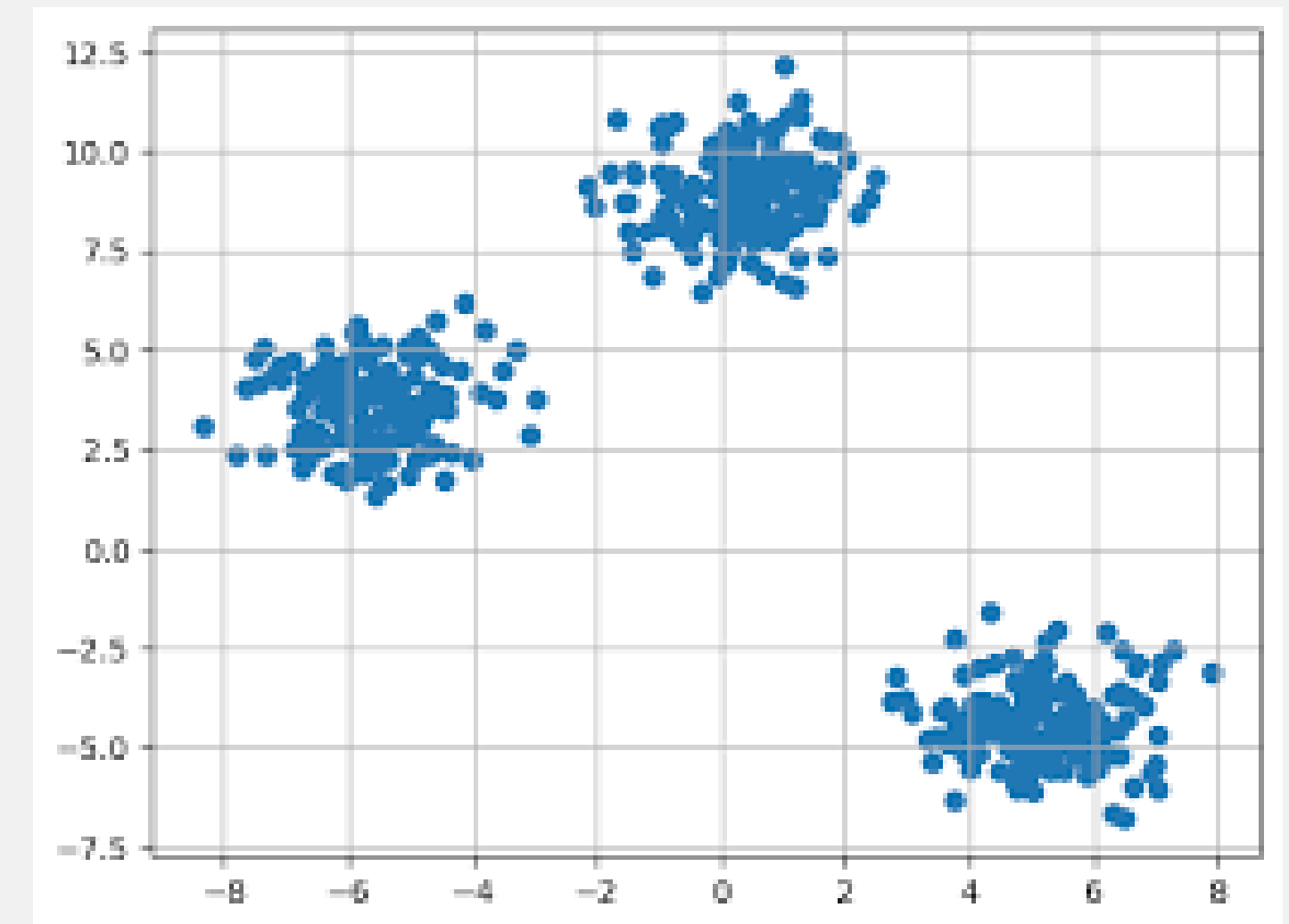
Dimensionality Reduction

- PCA
 - Non-negative matrix factorization
 - locally linear embedding
 - Isomap
- 

Clustering

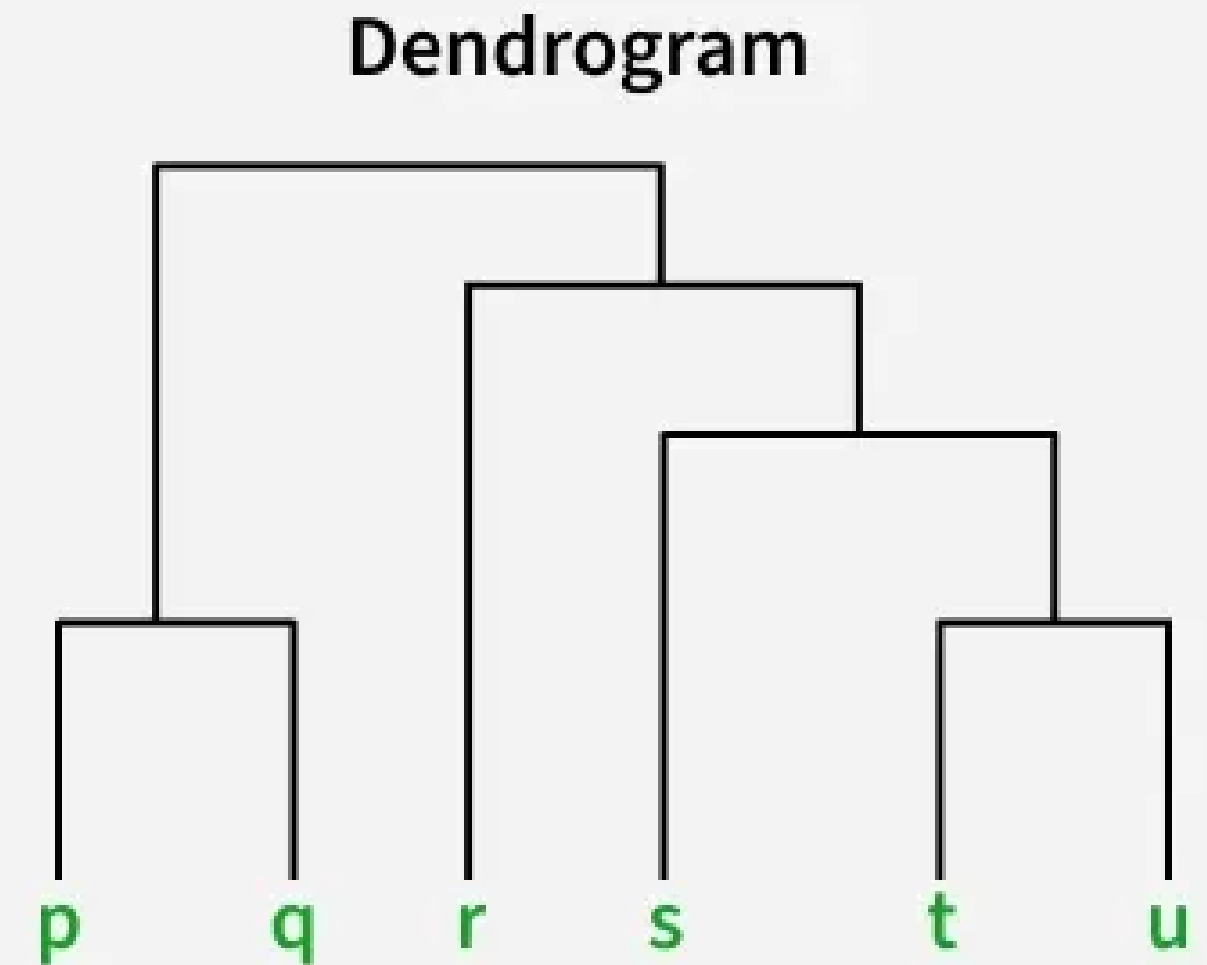
K means clustering

1. Choose the number of clusters K
2. Initialize K centroids randomly from the data
3. Compute the distance between each data point and all centroids
4. Assign each data point to the nearest centroid
5. Recalculate centroids as the mean of assigned points
6. Repeat steps 3–5 until centroids stop changing (convergence)
7. Output the final clusters and their centroids



Hierarchical clustering

- At the bottom of the dendrogram the points P, Q, R, S and T are all separate.
- As we move up, the closest points are merged into a single group.
- The lines connecting the points show how they are progressively merged based on similarity.
- The height at which they are connected shows how similar the points are to each other; the shorter the line the more similar they are

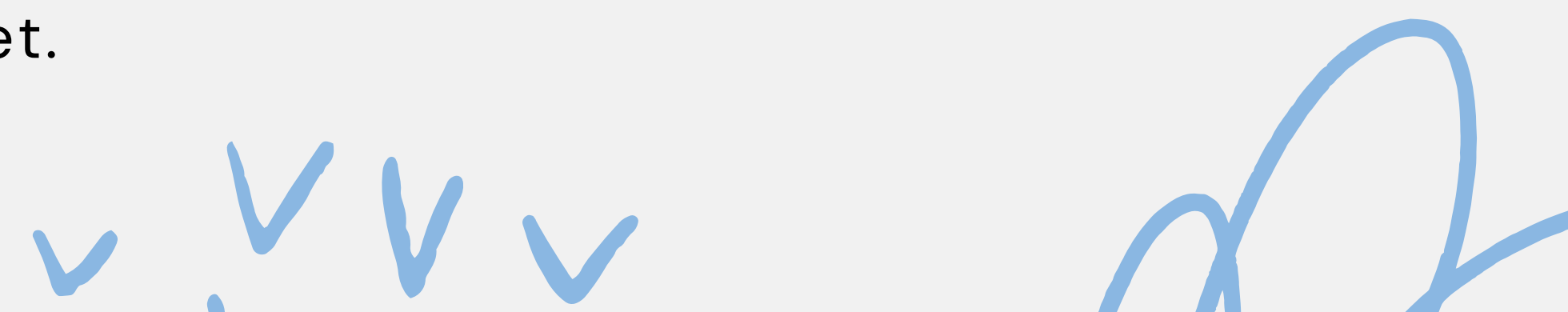


Hierarchical clustering – Agglomerative

1. **Start with individual points:** Each data point is its own cluster. For example if we have 5 data points we start with 5 clusters each containing just one data point.
2. **Calculate distances between clusters:** Calculate the distance between every pair of clusters. Initially since each cluster has one point this is the distance between the two data points.
3. **Merge the closest clusters:** Identify the two clusters with the smallest distance and merge them into a single cluster.
4. **Update distance matrix:** After merging we now have one less cluster. Recalculate the distances between the new cluster and the remaining clusters.
5. **Repeat steps 3 and 4:** Keep merging the closest clusters and updating the distance matrix until we have only one cluster left.

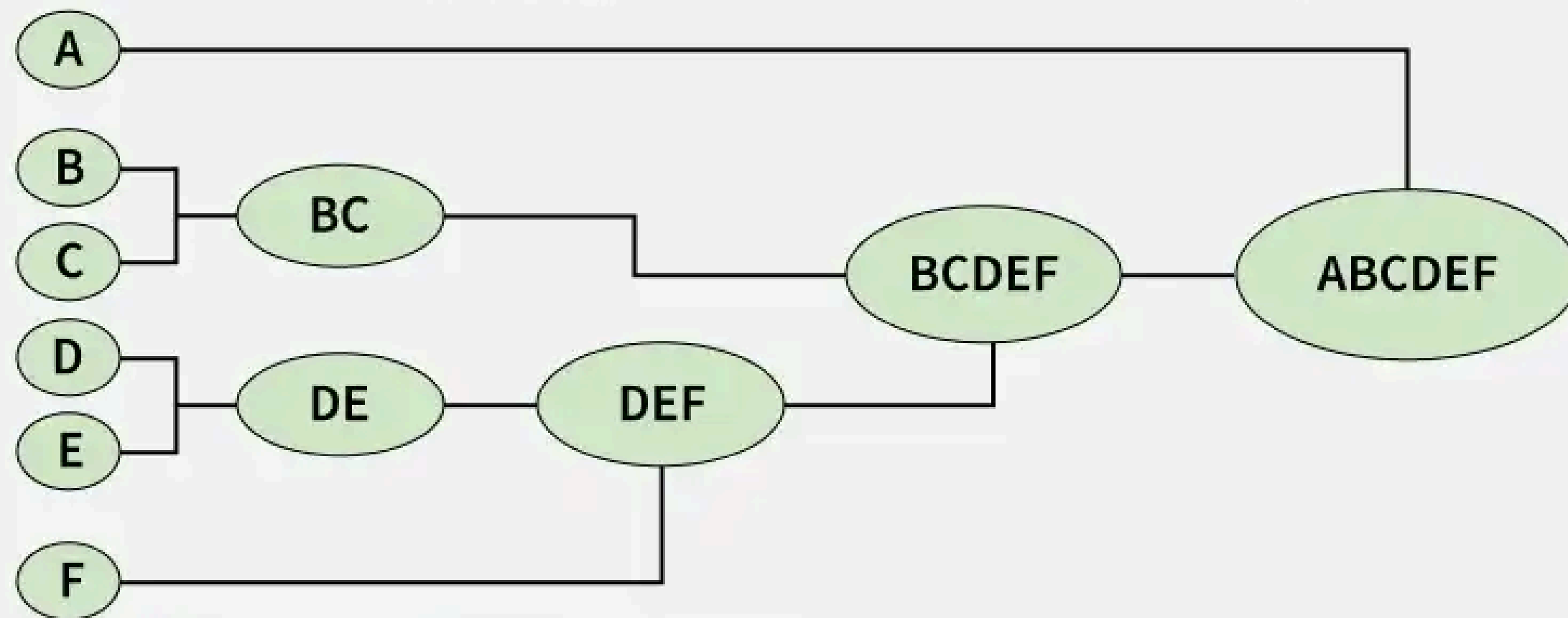


Hierarchical clustering – Divisive

1. **Start with all data points in one cluster:** Treat the entire dataset as a single large cluster.
 2. **Split the cluster:** Divide the cluster into two smaller clusters. The division is typically done by finding the two most dissimilar points in the cluster and using them to separate the data into two parts.
 3. **Repeat the process:** For each of the new clusters, repeat the splitting process: Choose the cluster with the most dissimilar points and split it again into two smaller clusters.
 4. **Stop when each data point is in its own cluster:** Continue this process until every data point is its own cluster or the stopping condition (such as a predefined number of clusters) is met.
- 

Top-Down (Divisive Clustering)

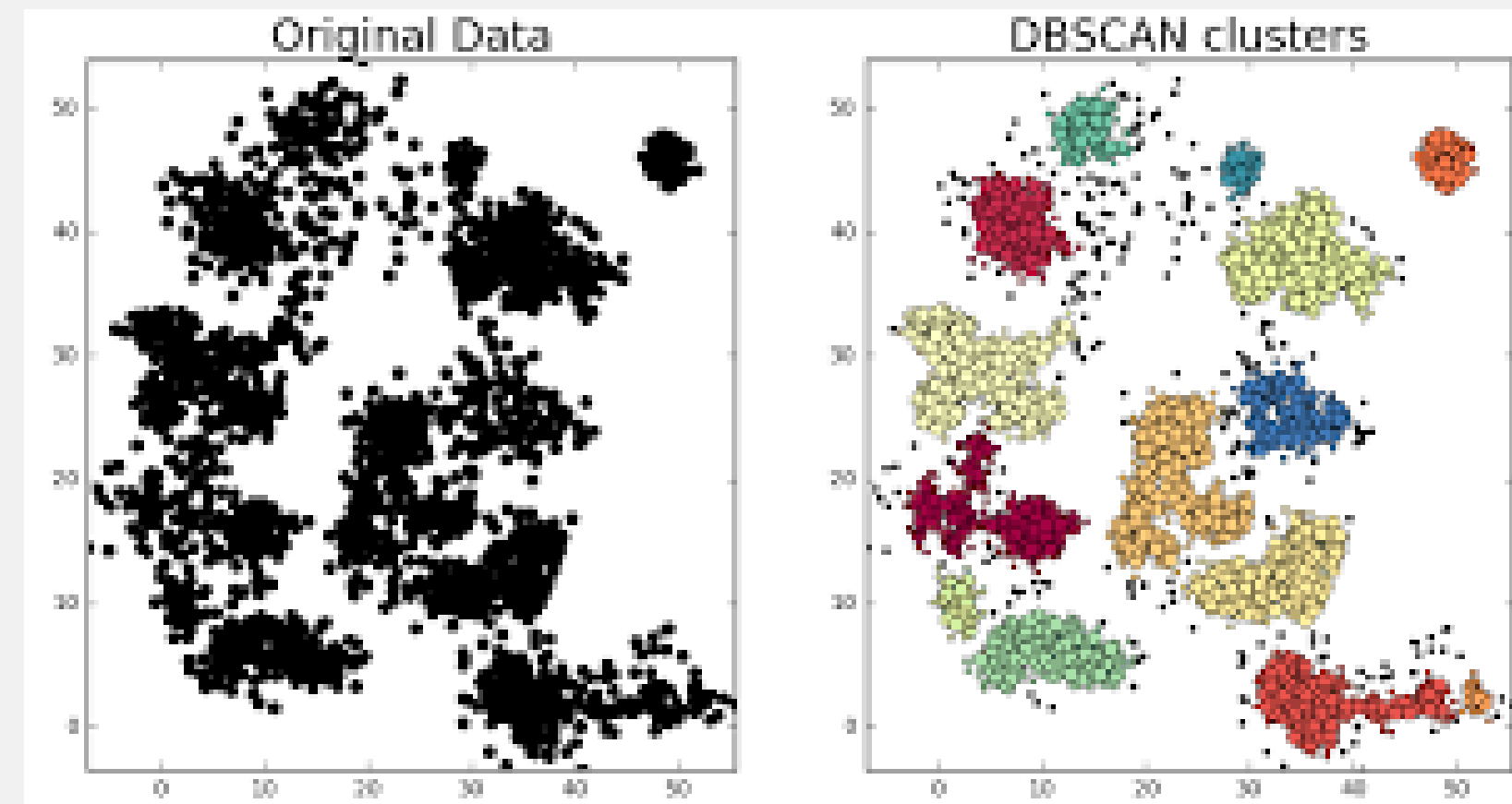
Bottom-Up (Agglomerative Clustering)



Density based clustering (DBSCAN)

Unlike K-Means or hierarchical clustering which assumes clusters are compact and spherical, DBSCAN perform well in handling real-world data irregularities such as:

- Arbitrary-Shaped Clusters: Clusters can take any shape not just circular or convex.
- Noise and Outliers: It effectively identifies and handles noise points without assigning them to any cluster.



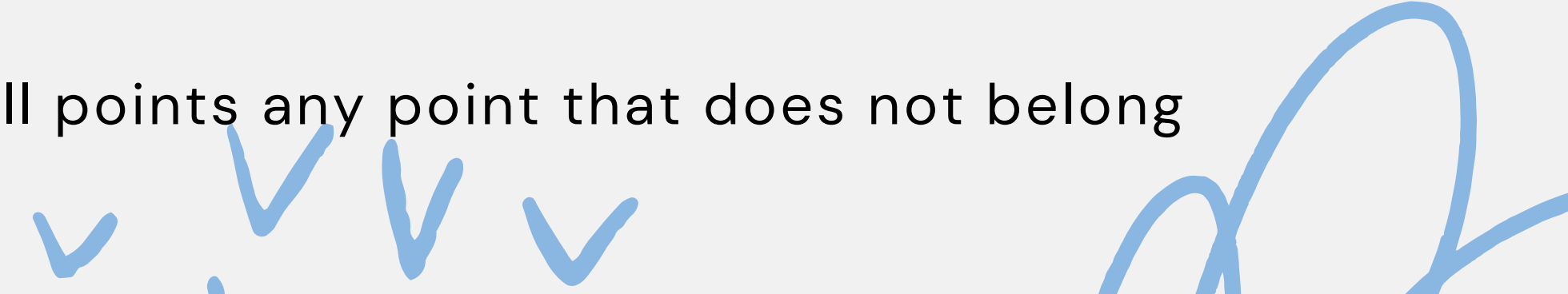
Density based clustering (DBSCAN)

key parameters:

- **eps:** This defines the radius of the neighborhood around a data point. If the distance between two points is less than or equal to eps they are considered neighbors.
- **MinPts:** This is the minimum number of points required within the eps radius to form a dense region. A general rule of thumb is to set $\text{MinPts} \geq D+1$ where D is the number of dimensions in the dataset. For most cases a minimum value of $\text{MinPts} = 3$ is recommended.



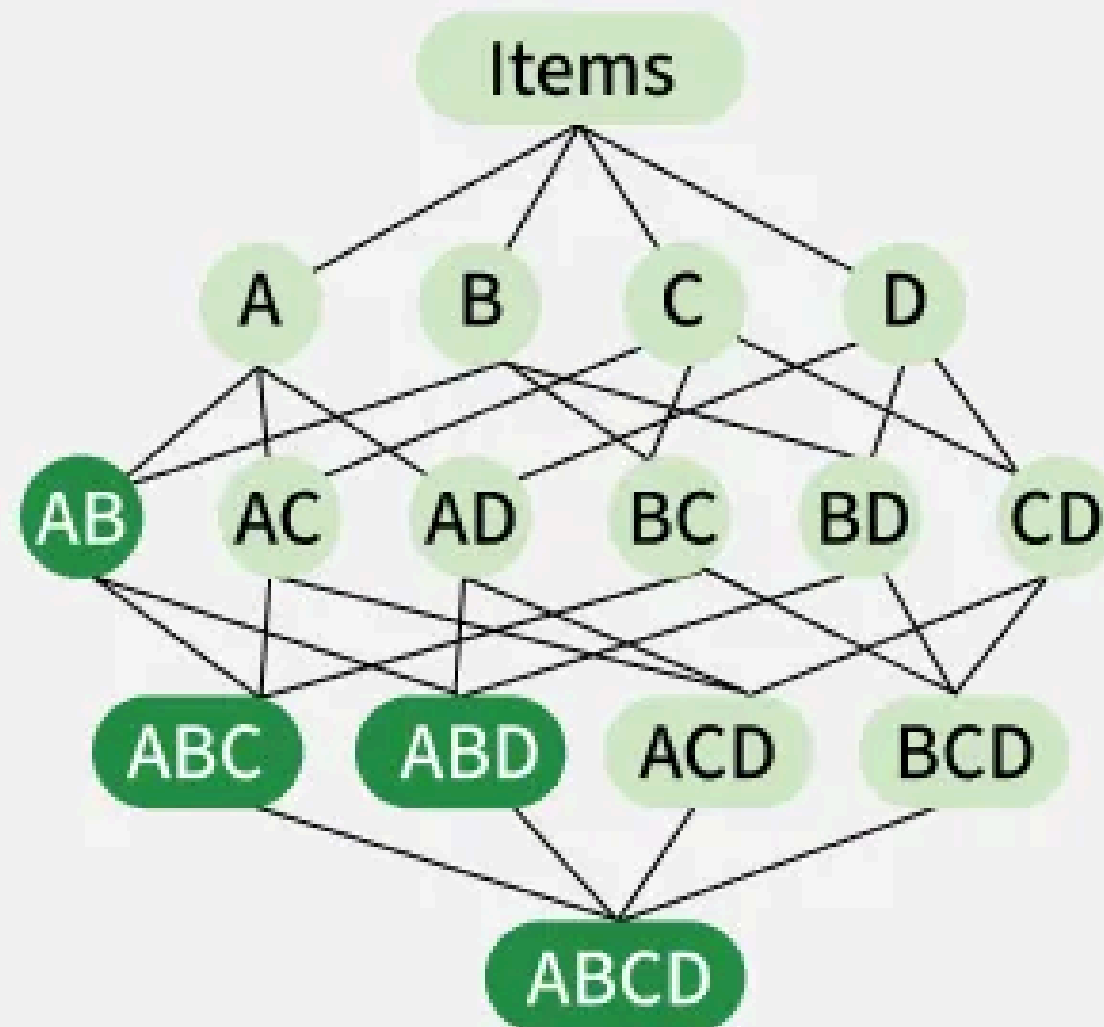
Density based clustering (DBSCAN)

1. **Identify Core Points:** For each point in the dataset count the number of points within its ϵ neighborhood. If the count meets or exceeds MinPts mark the point as a core point.
 2. **Form Clusters:** For each core point that is not already assigned to a cluster create a new cluster. Recursively find all density-connected points i.e points within the ϵ radius of the core point and add them to the cluster.
 3. **Density Connectivity:** Two points a and b are density-connected if there exists a chain of points where each point is within the ϵ radius of the next and at least one point in the chain is a core point. This chaining process ensures that all points in a cluster are connected through a series of dense regions.
 4. **Label Noise Points:** After processing all points any point that does not belong to a cluster is labeled as noise.
- 



Association Rule Learning

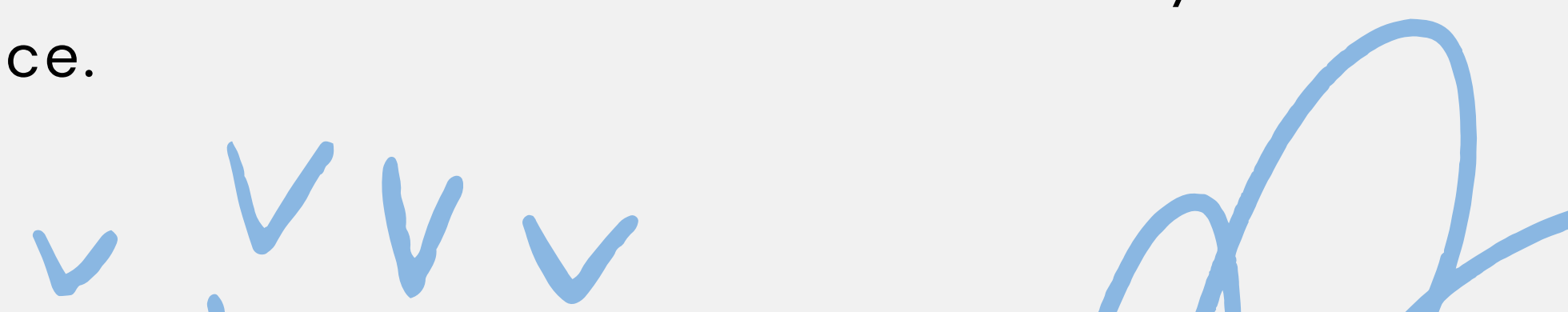
Apriori algorithm





Apriori algorithm

Key Metrics:

- **Support:** This metric measures how frequently an item appears in the dataset relative to the total number of transactions. A higher support indicates a more significant presence of the Item-Set in the dataset.
 - **Confidence:** Confidence assesses the likelihood that an item Y is purchased when item X is purchased. It provides insight into the strength of the association between two items.
 - **Lift:** Lift evaluates how much more likely two items are to be purchased together compared to being purchased independently. A lift greater than 1 suggests a strong positive association. Lift shows how strong the connection is between items. Like Bread and butter are much more likely to be bought together than by chance.
- 

Apriori algorithm

$$\text{Support}(A) = \frac{\text{Number of transactions containing itemset } A}{\text{Total number of transactions}}$$

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

Lift > 1 implies a positive association — items occur together more than expected.

Lift = 1 implies independence.

Lift < 1 implies a negative association.

Apriori algorithm

Transaction ID	Items Bought
T1	Bread, Butter, Milk
T2	Bread, Butter
T3	Bread, Milk
T4	Butter, Milk
T5	Bread, Milk

Threshold = 50%

Apriori algorithm

Step 1: Frequent 1-Itemsets (L1)

Count support:

- Bread $\rightarrow 4/5 = 80\%$ ✓
- Butter $\rightarrow 3/5 = 60\%$ ✓
- Milk $\rightarrow 4/5 = 80\%$ ✓

L1 = {Bread, Butter, Milk}

Step 2: Candidate 2-Itemsets (C2)

All possible pairs:

- (Bread, Butter)
- (Bread, Milk)
- (Butter, Milk)

Count support:

- (Bread, Butter) $\rightarrow T1, T2 \rightarrow 2/5 = 40\%$ ✗
- (Bread, Milk) $\rightarrow T1, T3, T5 \rightarrow 3/5 = 60\%$ ✓
- (Butter, Milk) $\rightarrow T1, T4 \rightarrow 2/5 = 40\%$ ✗

L2 = {Bread, Milk}

Apriori algorithm

Step 3: Candidate 3-Itemsets (C3)

Try to generate:

- (Bread, Butter, Milk)

Check support:

- Appears only in T1 $\rightarrow 1/5 = 20\%$ ✗

No itemsets meet threshold

Stop here (no L3)

Step 4: Generate Association Rules

From the only frequent 2-itemset:

{Bread, Milk}

Rule 1: Bread $\rightarrow$ Milk

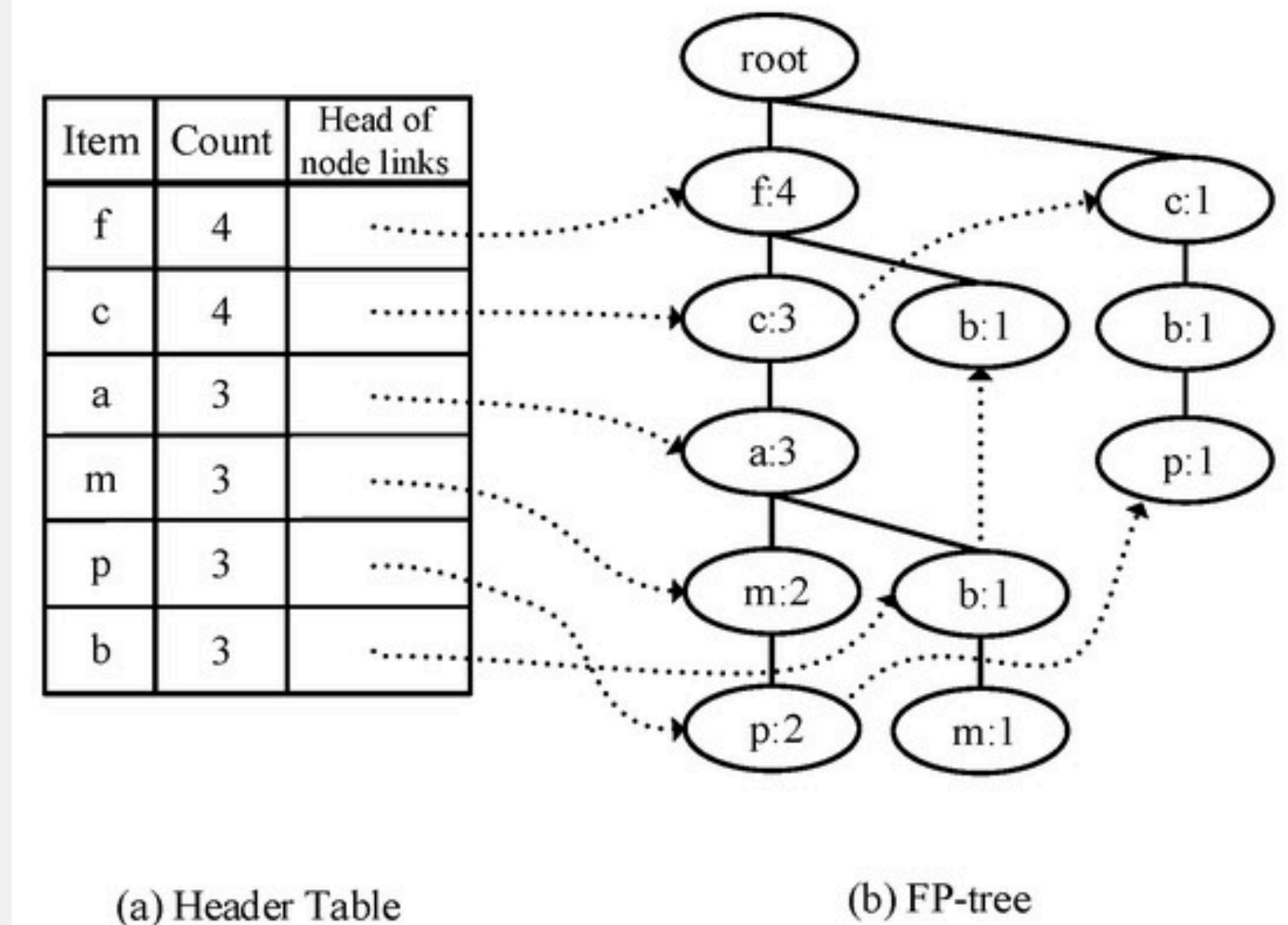
Confidence = $\text{support}(\text{Bread} \cup \text{Milk}) / \text{support}(\text{Bread})$
 $= 3/4 = 75\%$

Rule 2: Milk $\rightarrow$ Bread

Confidence = $\text{support}(\text{Bread} \cup \text{Milk}) / \text{support}(\text{Milk})$
 $= 3/4 = 75\%$

FP-growth algorithm

Unlike the Apriori algorithm which suffers from high computational cost due to candidate generation and multiple database scans. FP-Growth avoids these inefficiencies by compressing the data into an FP-Tree (Frequent Pattern Tree) and extracts patterns directly from it.



FP-growth algorithm

Transaction ID	Sorted Items by Frequency
T1	Bread, Milk, Butter
T2	Bread , Butter
T3	Bread, Milk
T4	Milk , Butter
T5	Bread

Assume minimum support = 50%

Total transactions = 5

Minimum support count = 3

FP-growth algorithm

Step 1: Count Frequency of Each Item

- Bread $\rightarrow$ 4
- Milk $\rightarrow$ 3
- Butter $\rightarrow$ 3

All items meet minimum support.

Step 2: Sort Items by Frequency

Descending order of frequency:

1. Bread (4)
2. Milk (3)
3. Butter (3)

Step 3: Sort Transactions According to Frequency Order

Reorder items inside each transaction using the global frequency order.

- T1 $\rightarrow$ Bread, Milk, Butter
- T2 $\rightarrow$ Bread, Butter
- T3 $\rightarrow$ Bread, Milk
- T4 $\rightarrow$ Milk, Butter
- T5 $\rightarrow$ Bread

FP-growth algorithm

Step 4: Construct the FP-Tree

Start with a NULL root.

Insert T1: Bread → Milk → Butter

- Create Bread:1
- Create Milk:1 under Bread
- Create Butter:1 under Milk

Insert T2: Bread → Butter

- Bread already exists → increment to 2
- Create Butter:1 under Bread

Insert T3: Bread → Milk

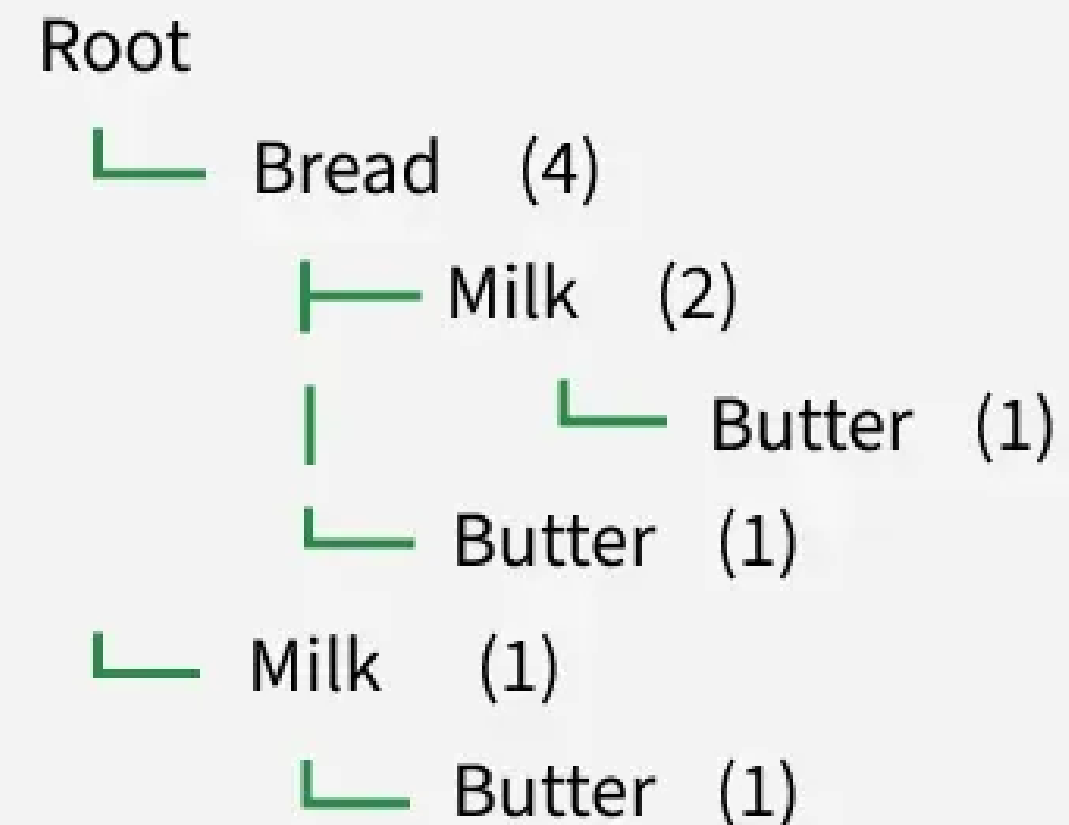
- Bread count becomes 3
- Milk under Bread count becomes 2

Insert T4: Milk → Butter

- Create separate Milk:1 from root
- Create Butter:1 under this Milk

Insert T5: Bread

- Bread count becomes 4



FP-growth algorithm

Step 6: Generate Conditional Pattern Bases

For Butter

Paths leading to Butter:

- Bread $\rightarrow$ Milk : 1
- Bread : 1
- Milk : 1

Conditional pattern base of Butter:

- {Bread, Milk} : 1
- {Bread} : 1
- {Milk} : 1

No item reaches support count 3.

No larger frequent pattern with Butter.

For Milk

Paths leading to Milk:

- Bread : 2

Conditional pattern base:

- {Bread} : 2

Support < 3

No larger frequent pattern.

FP-growth algorithm

Step 7: Frequent Patterns

Frequent 1-itemsets:

- Bread (4)
- Milk (3)
- Butter (3)

Frequent 2-itemsets:

- Bread, Milk (2) ✗
- Bread, Butter (2) ✗
- Milk, Butter (2) ✗

No 2-itemset satisfies minimum support 3.

Final Result

Frequent Itemsets:

- {Bread}
- {Milk}
- {Butter}

No larger frequent itemsets are found.